PS-033: DNS-Based Discovery of Diagnostic Services

DNS Side: Signed Zone and Validating Resolver (Student A)

Purpose

The DNS side publishes the three diagnostic services (latency, traceroute, loss/jitter) as records in a DNSSEC-signed zone, `lab.test`, so that the discovery client can find them by name instead of by hardcoded address. It also runs the validating resolver that the client queries, so that forged or altered answers are refused before they reach the client. This side answers one question only: whether a published record is authentic. It never decides whether a caller is allowed to use a service; that stays with the service itself.

Components

Component	What it does	Address
Authoritative server	BIND 9.20 in Docker. Holds and signs the `lab.test` zone.	`127.0.0.1:5300`
Validating resolver	Unbound with the zone's trust anchor. The client queries this, never BIND directly.	`127.0.0.1:5301`
Zone files and config	Zone file, BIND config, Unbound config and compose file, kept in the team GitHub repository.	GitHub repo

BIND is published on host port 5300 rather than 5353 or 53. macOS already uses UDP 5353 for Bonjour (mDNS) and often uses port 53, so either would clash. Inside the container BIND still listens on 53, and the port mapping is 5300:53 for both UDP and TCP. TCP matters because signed answers are larger and can fall back to TCP.

Publishing flow

The zone was built in stages so that each stage could be tested before the next one was added.

1. **Unsigned zone.** BIND serves `lab.test` with an NS record for `ns1` and A records for `latency`, `traceroute` and `lossjitter`, all pointing to 127.0.0.1. The SOA answer is checked for the AA (authoritative answer) flag.
2. **Service records.** Each service name gets an SVCB record carrying its port (8081, 8082, 8083), an address hint, and the SHA-256 digest of its descriptor in the private-use key `key65400`. This is the format agreed with Student B.
3. **Signing.** The zone is signed with DNSSEC, so every record set gets an RRSIG and the zone publishes its DNSKEY.
4. **Trust anchor and resolver.** The public trust anchor is exported and loaded into Unbound, which then sets the AD (Authenticated Data) flag on answers it has validated.
5. **Faulty entries.** Two deliberately faulty names are published for the client's negative tests (see below).

Service records

Name	Port	Record
`latency.lab.test`	8081	SVCB with port, address hint and descriptor digest
`traceroute.lab.test`	8082	SVCB with port, address hint and descriptor digest
`lossjitter.lab.test`	8083	SVCB with port, address hint and descriptor digest

Each record follows this layout, shown here for the latency service:

```
latency IN SVCB 1 . port=8081 ipv4hint=127.0.0.1 key65400="f9ae09f2df370aa461cf6ca43410a
f79fdef6b95c36fc62871d6be1b6b54d250"
```

A small JSON index lists the three services so that the client has a single starting point. For each service it gives the name, host, port and descriptor path (`/.well-known/descriptor.json`). The index is a convenience for discovery only. The client still validates each service's own record before trusting any value from it.

Deliberately faulty entries

Two extra names exist only so the client's failure handling can be tested against real DNS behaviour, not only simulated cases.

Name	What is wrong	What the client should report
`bogus.lab.test`	The chain of trust is broken, so validation fails. Unbound returns SERVFAIL.	BOGUS
`malformed.lab.test`	Correctly signed SVCB record, but it has no port.	MALFORMED

Separation of integrity and authorization

DNSSEC proves that a record came from the zone owner and was not changed on the way. It does not prove that the service behind the record is safe, and it does not give anyone permission to call it. The DNS side therefore publishes no tokens and no access rules. Each service checks the caller's token on every request, so finding an endpoint through DNS never grants access to it.



Testing

Each stage is checked with `dig`. Port 5300 queries BIND directly, and port 5301 goes through the validating resolver.

```
# Authoritative answers (BIND)
dig @127.0.0.1 -p 5300 latency.lab.test A +short
dig @127.0.0.1 -p 5300 lab.test SOA
dig @127.0.0.1 -p 5300 lab.test SOA +tcp +short

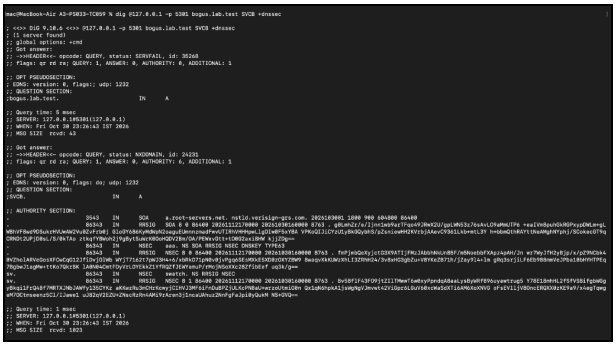
# Validated answers (Unbound)
dig @127.0.0.1 -p 5301 latency.lab.test SVCB +dnssec
dig @127.0.0.1 -p 5301 bogus.lab.test SVCB +dnssec
```

Check	Expected	Result
Zone loads in BIND	Log shows the zone loaded with its serial	PASS
A record for each service name	127.0.0.1	PASS
SOA over UDP	NOERROR, aa flag set	PASS
SOA over TCP	Same answer as UDP	PASS
SVCB through Unbound (latency)	ad flag set, port 8081	PASS
SVCB through Unbound (traceroute)	ad flag set, port 8082	PASS
SVCB through Unbound (lossjitter)	ad flag set, port 8083	PASS
Faulty zone: bogus	SERVFAIL, no ad flag	PASS
Faulty zone: malformed	Valid signature, ad flag set, no port in record	PASS

Result: 9 of 9 checks passed.



Validated query (latency.lab.test) via Unbound showing NOERROR and the ad (Authenticated Data) flag.



Faulty zone query (bogus.lab.test) via Unbound correctly refused with SERVFAIL (no ad flag).

Scope against the proposal

The proposal (Optimization slide) lists several optimisations. Status of the DNS side in the submitted prototype:

- **Signed, dynamic discovery:** implemented. The three services are published as SVCB records in a signed zone, so moving an endpoint means updating one record and raising the zone serial. No client needs editing.
- **DNSSEC-validated answers, bogus data rejected:** implemented. Unbound validates the chain of trust and refuses `bogus.lab.test` with `SERVFAIL`.
- **Metadata the client can verify:** implemented. Each record carries the port, an address hint and the descriptor digest, and the malformed entry shows that a signed record can still be unusable.
- **TLSA records and priority fallback:** not implemented; see Future work.

Future work

The following items were deferred to keep the prototype focused on discovery, validation and the failure cases:

- **TLSA/DANE:** publish a TLSA record for each service so the client can pin the service's TLS key through DNS. This needs the services to run over TLS.
- **Priority fallback:** publish more than one SVCB record per service with different priorities, so the client can try a second endpoint when the first is down. The zone currently has one record per service.
- **Key management:** add key rollover and, for a real domain, a DS record in the parent zone instead of a hand-installed trust anchor.
- **Registered SVCB parameter:** replace the private-use key with a registered one.

Other limitations

- The zone is signed with a single locally generated key pair. Because `lab.test` is not in the real DNS root, the trust anchor is installed in Unbound by hand.
- Every service name points to 127.0.0.1 and the services are told apart by port. In a real deployment each service would have its own address.
- Records have a 300 second TTL. After a change, resolvers may serve the old answer until the TTL expires.
- The prototype serves the services over plain HTTP. Authenticity of the metadata comes from DNSSEC and the digest, not from TLS.